

The system implements a **5-step pipeline** designed to automate the verification workflow for hardware engineers:

1. **Requirement Intake:** Accepts natural language descriptions and corresponding Verilog code.
2. **Test Pattern Generation:** Uses an LLM to generate intelligent test patterns covering corner cases, boundary values, and random sequences.
3. **Golden Model Creation:** Automatically generates a Python-based reference (golden) model from the natural language description to calculate expected outputs.
4. **Verification Logic Integration:** Updates the Verilog testbench with automated pass/fail checking logic based on the golden model's results.
5. **Simulation & Reporting:** Executes the simulation (e.g., via Icarus Verilog) and generates a detailed test summary.

Key Components and Technical Implementation

LLM Client (LLMClient)

The core of the system is a unified interface for interacting with LLM APIs. It supports providers like OpenAI and is configured to handle complex prompts with specific system instructions to ensure high-quality code generation.

Testbench Generator (TestbenchGenerator)

This module extracts module metadata (ports, names) from Verilog code and prompts the LLM to create a "skeleton" testbench. The LLM is instructed to focus on comprehensive coverage, including:

- Corner cases and boundary values.
- Systematic numbering of test cases.
- Input signal displays for debugging.

Golden Model Generator (GoldenModelGenerator)

A unique feature of this system is the generation of a **Python Golden Model**. By converting hardware logic into executable Python functions, the system can programmatically determine the "correct" answer for any given input pattern without requiring manual entry from the user.

Testbench Updater (TestbenchUpdater)

The final stage involves injecting verification logic into the generated testbench. It adds:

- **Result Tracking:** Variables like `passed_tests` and `failed_tests`.
- **Comparison Logic:** Automated checks that compare actual simulation outputs against the golden outputs.
- **Visual Feedback:** Uses symbols like "✓" and "X" for clear reporting.

Features & Benefits

- **Automation:** Reduces the manual effort required to write complex verification environments.
- **Accuracy:** Uses a secondary Python implementation (Golden Model) to verify the primary Verilog logic.
- **Self-Contained:** The provided environment is ready-to-run, requiring only standard tools like iverilog and an OpenAI API key.
- **Intelligent Coverage:** Leverages GPT-4's reasoning to identify edge cases that might be overlooked by human designers.

```
... All Generated Files:
=====
.config/
  .last_opt_in_prompt.yaml          (3 bytes)
  .last_survey_prompt.yaml          (36 bytes)
  .last_update_check.json           (135 bytes)
  active_config                     (7 bytes)
  config_sentinel                   (0 bytes)
  configurations/
    config_default                  (94 bytes)
  default_configs.db                (12,288 bytes)
  gce                               (5 bytes)
  hidden_gcloud_config_universe_descriptor_data_cache_configs.db (12,288 bytes)
  logs/
    2026.01.16/
      14.23.31.981136.log            (50,803 bytes)
      14.24.03.314209.log            (465 bytes)
      14.24.13.071214.log            (12,656 bytes)
      14.24.18.954466.log            (465 bytes)
      14.24.28.646070.log            (704 bytes)
      14.24.29.392089.log            (699 bytes)
```

```
LLM-aided-Testbench-Generation/
.git/
  HEAD                                (21 bytes)
  branches/
  config                              (284 bytes)
  description                          (73 bytes)
  hooks/
    applypatch-msg.sample             (478 bytes)
    commit-msg.sample                 (896 bytes)
    fsmonitor-watchman.sample         (4,655 bytes)
    post-update.sample                (189 bytes)
    pre-applypatch.sample              (424 bytes)
    pre-commit.sample                (1,643 bytes)
    pre-merge-commit.sample           (416 bytes)
    pre-push.sample                  (1,374 bytes)
    pre-rebase.sample                 (4,898 bytes)
    pre-receive.sample                (544 bytes)
    prepare-commit-msg.sample          (1,492 bytes)
    push-to-checkout.sample            (2,783 bytes)
    update.sample                     (3,650 bytes)
  index                              (2,387 bytes)
  info/
    exclude                          (240 bytes)
  logs/
    HEAD                              (207 bytes)
  refs/
    heads/
      main                            (207 bytes)
    remotes/
      origin/
        HEAD                          (207 bytes)
  objects/
    info/
    pack/
```

main.py	(5,640 bytes)
requirements.txt	(15 bytes)
src/	
__init__.py	(423 bytes)
golden_model_generator.py	(5,703 bytes)
llm_client.py	(2,296 bytes)
testbench_generator.py	(5,966 bytes)
testbench_pipeline.py	(7,891 bytes)
testbench_updater.py	(11,525 bytes)
tests/	
__init__.py	(54 bytes)
test_golden_model.py	(2,855 bytes)
test_module_extraction.py	(2,152 bytes)
test_pipeline.py	(2,858 bytes)
adder/	
golden_model.py	(340 bytes)
sim.vvp	(21,176 bytes)
test_patterns_with_golden.json	(994 bytes)
testbench_final.v	(10,349 bytes)
testbench_initial.v	(1,644 bytes)
adder4bit.v	(241 bytes)
binary_to_bcd/	
golden_model.py	(509 bytes)
sim.vvp	(11,575 bytes)
test_patterns_with_golden.json	(876 bytes)
testbench_final.v	(5,789 bytes)
testbench_initial.v	(2,103 bytes)
binary_to_bcd_converter.v	(860 bytes)
mux/	
golden_model.py	(79 bytes)
sim.vvp	(10,286 bytes)
test_patterns_with_golden.json	(922 bytes)
testbench_final.v	(4,269 bytes)
testbench_initial.v	(1,556 bytes)
mux2to1.v	(134 bytes)
sample_data/	
README.md	(962 bytes)
anscombe.json	(1,697 bytes)
california_housing_test.csv	(301,141 bytes)

```
LLM-aided-Testbench-Generation/
.git/
  HEAD                                (21 bytes)
  branches/
  config                              (284 bytes)
  description                          (73 bytes)
  hooks/
    applypatch-msg.sample             (478 bytes)
    commit-msg.sample                 (896 bytes)
    fsmonitor-watchman.sample         (4,655 bytes)
    post-update.sample                (189 bytes)
    pre-applypatch.sample              (424 bytes)
    pre-commit.sample                (1,643 bytes)
    pre-merge-commit.sample            (416 bytes)
    pre-push.sample                   (1,374 bytes)
    pre-rebase.sample                 (4,898 bytes)
    pre-receive.sample                (544 bytes)
    prepare-commit-msg.sample          (1,492 bytes)
    push-to-checkout.sample            (2,783 bytes)
    update.sample                     (3,650 bytes)
  index                              (2,387 bytes)
  info/
    exclude                          (240 bytes)
  logs/
    HEAD                              (207 bytes)
  refs/
    heads/
      main                            (207 bytes)
    remotes/
      origin/
        HEAD                          (207 bytes)
  objects/
    info/
    pack/
```